

# Linear Algebra Cheat Sheet

A compact reference for the maths-refresh notebooks so far.

This is not meant to replace the notebooks. It is the “what did we mean by that again?” sheet.

---

## 1. Vectors

A **vector** is an ordered collection of numbers.

In this course-prep work, a vector can mean several related things:

- a point in space, such as  $(3, 2)$
- a movement from the origin to a point
- a direction with a length
- a row or column of data used in a calculation

For a 2D vector:

$$\mathbf{v} = \begin{bmatrix} x \\ y \end{bmatrix}$$

$x$  tells us the horizontal component and  $y$  tells us the vertical component.

### NumPy shape matters

A 1D NumPy array:

```
v = np.array([3, 2])  
v.shape
```

has shape:

```
(2,)
```

A column-shaped vector:

```
v_col = np.array([[3], [2]])  
v_col.shape
```

has shape:

```
(2, 1)
```

They contain similar-looking numbers, but they are not the same shape. Shape matters when doing matrix multiplication.

---

## 2. Vector length / norm

The **norm** of a vector is its length.

For:

$$\mathbf{v} = \begin{bmatrix} x \\ y \end{bmatrix}$$

its length is:

$$|\mathbf{v}| = \sqrt{x^2 + y^2}$$

Example:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$|\mathbf{v}| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

In NumPy:

```
np.linalg.norm(v)
```

---

## 3. Unit vectors

A **unit vector** has length 1.

It keeps the direction of a vector but removes the scale.

To turn a vector into a unit vector:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{|\mathbf{v}|}$$

The hat usually means “unit version of”.

Example:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Since  $|\mathbf{v}| = 5$ :

$$\hat{\mathbf{v}} = \frac{1}{5} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

## 4. Dot product

The **dot product** combines two vectors and produces a single number.

For two 2D vectors:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

The dot product is:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2$$

In NumPy:

```
np.dot(a, b)
```

or:

```
a @ b
```

### Geometric meaning

The dot product measures how much two vectors point in the same direction.

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos(\theta)$$

where  $\theta$  is the angle between them.

### Dot product signs

| Dot product | Meaning                                    |
|-------------|--------------------------------------------|
| positive    | vectors point partly in the same direction |

| Dot product | Meaning                                     |
|-------------|---------------------------------------------|
| zero        | vectors are perpendicular / orthogonal      |
| negative    | vectors point partly in opposite directions |

## Maximum and minimum dot product

For fixed vector lengths:

$$-|\mathbf{a}||\mathbf{b}| \leq \mathbf{a} \cdot \mathbf{b} \leq |\mathbf{a}||\mathbf{b}|$$

The maximum happens when the vectors point in exactly the same direction.

The minimum happens when they point in exactly opposite directions.

## 5. Cosine similarity

**Cosine similarity** is the dot product after removing the effect of vector length.

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|}$$

It always lies between  $-1$  and  $1$ .

| Cosine similarity | Meaning                             |
|-------------------|-------------------------------------|
| $1$               | same direction                      |
| $0$               | perpendicular / unrelated direction |
| $-1$              | opposite direction                  |

This is useful when direction matters more than raw magnitude.

## 6. Projection

A **projection** answers this question:

How much of one vector lies in the direction of another vector?

The projection of  $\mathbf{a}$  onto  $\mathbf{b}$  is:

$$\text{proj}_{\mathbf{b}}(\mathbf{a}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{b} \cdot \mathbf{b}} \mathbf{b}$$

This gives a vector pointing along  $\mathbf{b}$ .

### Projection using a unit vector

If  $\hat{\mathbf{b}}$  is the unit version of  $\mathbf{b}$ , then:

$$\text{proj}_{\mathbf{b}}(\mathbf{a}) = (\mathbf{a} \cdot \hat{\mathbf{b}}) \hat{\mathbf{b}}$$

This can feel more intuitive:

1.  $\mathbf{a} \cdot \hat{\mathbf{b}}$  gives the signed length of  $\mathbf{a}$  in the  $\mathbf{b}$  direction.
2. Multiplying by  $\hat{\mathbf{b}}$  turns that signed length back into a vector.

---

## 7. Components

A vector can be split into:

- the part parallel to another vector
- the leftover part perpendicular to it

For vector  $\mathbf{a}$  projected onto  $\mathbf{b}$ :

$$\mathbf{a}_{\parallel} = \text{proj}_{\mathbf{b}}(\mathbf{a})$$

The perpendicular leftover is:

$$\mathbf{a}_{\perp} = \mathbf{a} - \mathbf{a}_{\parallel}$$

So:

$$\mathbf{a} = \mathbf{a}_{\parallel} + \mathbf{a}_{\perp}$$

The key check:

$$\mathbf{a}_{\perp} \cdot \mathbf{b} = 0$$

The leftover is perpendicular to the direction we projected onto.

---

## 8. Matrices

A **matrix** is a rectangular grid of numbers.

Example 2-by-2 matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

A matrix can represent a transformation.

For a vector:

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

matrix-vector multiplication gives:

$$A\mathbf{x} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

In NumPy:

```
A @ x
```

---

## 9. The identity matrix

The **identity matrix** leaves vectors unchanged.

In 2D:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Then:

$$I\mathbf{x} = \mathbf{x}$$

It is the matrix equivalent of multiplying by 1.

---

## 10. Basis vectors

The standard 2D basis vectors are:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Any 2D vector can be built from these:

$$\begin{bmatrix} x \\ y \end{bmatrix} = x\mathbf{e}_1 + y\mathbf{e}_2$$

So:

- $x$  says how much of the first basis vector we use
- $y$  says how much of the second basis vector we use

## 11. Matrix columns as transformed basis vectors

This is one of the most important ideas so far.

For:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The first column tells us where  $\mathbf{e}_1$  lands:

$$A\mathbf{e}_1 = \begin{bmatrix} a \\ c \end{bmatrix}$$

The second column tells us where  $\mathbf{e}_2$  lands:

$$A\mathbf{e}_2 = \begin{bmatrix} b \\ d \end{bmatrix}$$

So the matrix:

$$A = \begin{bmatrix} | & | \\ A\mathbf{e}_1 & A\mathbf{e}_2 \\ | & | \end{bmatrix}$$

A matrix transformation is fully described by where it sends the basis vectors.

Then any vector follows from that:

$$A\begin{bmatrix} x \\ y \end{bmatrix} = x(A\mathbf{e}_1) + y(A\mathbf{e}_2)$$

## 12. How the four 2-by-2 matrix elements behave

For:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

and:

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

we get:

$$\mathbf{A} \mathbf{x} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

So:

| Element | Main effect                                  |
|---------|----------------------------------------------|
| $a$     | how much original $x$ contributes to new $x$ |
| $b$     | how much original $y$ contributes to new $x$ |
| $c$     | how much original $x$ contributes to new $y$ |
| $d$     | how much original $y$ contributes to new $y$ |

Another way to read it:

$$\mathbf{A} = \begin{bmatrix} \text{new x from old x} & \text{new x from old y} \\ \text{new y from old x} & \text{new y from old y} \end{bmatrix}$$

---

## 13. Scaling

A scaling matrix stretches or shrinks space.

$$\mathbf{S} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Then:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_{xx} & s_{yy} \end{bmatrix}$$

Examples:

| Matrix                                             | Effect                  |
|----------------------------------------------------|-------------------------|
| $\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$     | doubles horizontal size |
| $\begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$     | triples vertical size   |
| $\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$ | halves everything       |

---

## 14. Reflection

A reflection flips space across an axis.

Reflect across the  $x$ -axis:

$$\begin{bmatrix} 1 & 0 & 0 & -1 \end{bmatrix}$$

Reflect across the  $y$ -axis:

$$\begin{bmatrix} -1 & 0 & 0 & 1 \end{bmatrix}$$

Reflect through the origin:

$$\begin{bmatrix} -1 & 0 & 0 & -1 \end{bmatrix}$$

---

## 15. Shear

A shear slants a shape while keeping some parallel structure.

Horizontal shear:

$$\begin{bmatrix} 1 & k & 0 & 1 \end{bmatrix}$$

This means:

$$x_{\text{new}} = x + ky$$

The higher the  $y$  value, the more the point is pushed horizontally.

Vertical shear:

$$\begin{bmatrix} 1 & 0 & k & 1 \end{bmatrix}$$

This means:

$$y_{\text{new}} = kx + y$$

The further right the point is, the more it is pushed vertically.

---

## 16. Rotation

A 2D rotation matrix for angle  $\theta$  is:

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Then:

$$R \mathbf{x}$$

rotates vector  $\mathbf{x}$  anticlockwise by  $\theta$ .

Special case: 90 degrees anticlockwise.

Since:

$$\cos 90^\circ = 0, \quad \sin 90^\circ = 1$$

we get:

$$R_{90} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

So:

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -y \\ x \end{pmatrix}$$

---

## 17. Composing transformations

To **compose** transformations means to apply one transformation after another.

If we first apply  $A$ , then apply  $B$ , the combined transformation is:

$$B(A\mathbf{x})$$

which can be written as:

$$(BA)\mathbf{x}$$

Important: matrix multiplication order matters.

Usually:

$$BA \neq AB$$

So “rotate then shear” may not give the same result as “shear then rotate”.

---

## 18. Linear transformations

A matrix transformation is a **linear transformation**.

It has two key properties:

### Additivity

$$A(\mathbf{u} + \mathbf{v}) = A\mathbf{u} + A\mathbf{v}$$

### Scaling compatibility

$$A(c\mathbf{v}) = c(A\mathbf{v})$$

Plain-English version:

A linear transformation preserves the “vector arithmetic structure” of space.

For 2D visual transformations, this means:

- grid lines stay straight
- parallel grid lines stay parallel
- the origin stays fixed

---

## 19. Linear combinations

A **linear combination** means “scaled things added together”.

For vectors:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n$$

where the  $c$  values are scalar weights.

Example:

$$3\mathbf{v}_1 - 2\mathbf{v}_2$$

is a linear combination of  $\mathbf{v}_1$  and  $\mathbf{v}_2$ .

Matrix-vector multiplication is built from linear combinations of matrix columns.

If:

$$A = \begin{bmatrix} | & | \\ \mathbf{a}_1 & \mathbf{a}_2 \\ | & | \end{bmatrix}$$

then:

$$A \begin{bmatrix} x \\ y \end{bmatrix} = x \mathbf{a}_1 + y \mathbf{a}_2$$

## 20. Span

The **span** of a set of vectors is all the points you can reach using linear combinations of those vectors.

For two vectors:

$$\operatorname{span}(\mathbf{v}_1, \mathbf{v}_2) = \{c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2\}$$

In 2D:

- two non-parallel vectors can span the whole plane
- two parallel vectors only span a line
- one non-zero vector spans a line

## 21. Column space

The **column space** of a matrix is the span of its columns.

If:

$$A = \begin{bmatrix} | & | \\ \mathbf{a}_1 & \mathbf{a}_2 \\ | & | \end{bmatrix}$$

then:

$$\operatorname{Col}(A) = \operatorname{span}(\mathbf{a}_1, \mathbf{a}_2)$$

Plain-English version:

The column space is the set of outputs the matrix can possibly produce.

For least squares, this matters because sometimes the target vector is not exactly reachable. In that case, we find the reachable vector that gets closest.

## 22. Systems of linear equations

A system of linear equations can be written as:

$$A\mathbf{x} = \mathbf{b}$$

where:

- $A$  is the matrix of known coefficients
- $\mathbf{x}$  is the vector of unknowns
- $\mathbf{b}$  is the target/output vector

Example:

$$\begin{aligned} 2m + c &= 5 \\ 3m + c &= 7 \end{aligned}$$

can be written as:

$$\begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

---

## 23. Overdetermined systems

An **overdetermined** system has more equations than unknowns.

For fitting a line:

$$y = mx + c$$

we may have many data points but only two unknowns:  $m$  and  $c$ .

That gives a system like:

$$X\boldsymbol{\beta} = \mathbf{y}$$

where:

$$\boldsymbol{\beta} = \begin{bmatrix} m \\ c \end{bmatrix}$$

Usually, there is no exact line that passes through every point.

So instead of solving exactly, we look for the best approximate solution.

---

## 24. Design matrix

A **design matrix** is a matrix whose rows describe the cases where we want predictions.

For a straight-line model:

$$y = mx + c$$

and input values:

$$x_1, x_2, \dots, x_n$$

we use:

$$X = \begin{bmatrix} x_1 & 1 & x_2 & 1 & \dots & \dots & x_n & 1 \end{bmatrix}$$

The first column is the  $x$  values.

The second column is all ones, because the intercept  $c$  is added once for every prediction.

Then:

$$X \boldsymbol{\beta} = \begin{bmatrix} x_1 & 1 & x_2 & 1 & \dots & \dots & x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} mx_1 + c & mx_2 + c & \dots & mx_n + c \end{bmatrix}$$

So the design matrix lets the model make many predictions at once.

Important wording:

In prediction/evaluation mode, the design matrix is not “training data” in the loose everyday sense. It is the structured set of cases at which we want the model expression evaluated.

---

## 25. Parameters / coefficients

For the straight-line model:

$$y = mx + c$$

$m$  and  $c$  are the model parameters.

We can collect them into a parameter vector:

$$\boldsymbol{\beta} = \begin{bmatrix} m \\ c \end{bmatrix}$$

Then predictions can be written compactly as:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}$$

The hat on  $\hat{\mathbf{y}}$  means “predicted version of  $\mathbf{y}$ ”.

---

## 26. Predictions

For a model:

$$y = mx + c$$

with parameters:

$$m = 2, \text{ \texttt{c} = 1}$$

and design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 & 1 & 3 & 1 \\ 2 & 1 & 3 & 5 & 7 \end{bmatrix}$$

we get:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta} = \begin{bmatrix} 1 & 1 & 2 & 1 & 3 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 & 5 & 7 \end{bmatrix}$$

This is the model evaluated at several  $x$  values in one matrix operation.

---

## 27. Residuals

A **residual** is the difference between the actual value and the predicted value.

Using the convention from the least-squares notebooks:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$$

or:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$$

For one point:

$$r_i = y_i - \hat{y}_i$$

A positive residual means the actual value is above the prediction.

A negative residual means the actual value is below the prediction.

---

## 28. Sum of squared residuals

Least squares uses squared residuals.

For residual vector:

$$\mathbf{r} = \begin{bmatrix} r_1 & r_2 & \dots & r_n \end{bmatrix}$$

The sum of squared residuals is:

$$\sum_{i=1}^n r_i^2$$

This is also:

$$|\mathbf{r}|^2$$

Squaring does two useful things:

- removes signs, so positive and negative errors do not cancel out
  - penalises larger errors more heavily
- 

## 29. Least squares

**Least squares** means choosing parameters that minimise the sum of squared residuals.

For a line:

$$y = mx + c$$

we choose  $m$  and  $c$  to make this as small as possible:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In matrix form:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$$

Plain-English version:

Find the parameter vector whose predictions get as close as possible to the actual target vector, using squared distance as the measure of closeness.

---

## 30. Least squares as projection

The prediction vector is:

$$\hat{\mathbf{y}} = X\boldsymbol{\beta}$$

Because  $\hat{\mathbf{y}}$  is produced by multiplying  $X$  by some parameter vector, it must live in the column space of  $X$ .

If the real target vector  $\mathbf{y}$  is not in the column space, we cannot reach it exactly.

So least squares finds the closest reachable vector:

$$\hat{\mathbf{y}}$$

This is the projection of  $\mathbf{y}$  onto the column space of  $X$ .

At the best fit, the residual vector is perpendicular to the column space:

$$X^T(\mathbf{y} - X\boldsymbol{\beta}) = \mathbf{0}$$

This is the gateway to the normal equation.

---

## 31. Normal equation

Starting from:

$$X^T(\mathbf{y} - X\boldsymbol{\beta}) = \mathbf{0}$$

expand:

$$X^T\mathbf{y} - X^TX\boldsymbol{\beta} = \mathbf{0}$$

so:

$$X^TX\boldsymbol{\beta} = X^T\mathbf{y}$$

This is the **normal equation**.

When it is safe to solve directly, we get:

$$\mathbf{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

In practice, we usually prefer numerical solvers rather than manually calculating the inverse.

In NumPy:

```
beta, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)
```

## 32. Transpose

The **transpose** of a matrix swaps rows and columns.

If:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$$

then:

$$A^T = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$$

In NumPy:

```
A.T
```

In least squares,  $X^T$  appears because the best-fit residual must be perpendicular to every column of  $X$ .

## 33. Orthogonal / perpendicular

Two vectors are **orthogonal** when their dot product is zero.

$$\mathbf{a} \cdot \mathbf{b} = 0$$

In 2D, this means they are perpendicular.

In higher dimensions, “orthogonal” is the more general word.

## 34. Common NumPy translations

| Maths                        | NumPy                                              |
|------------------------------|----------------------------------------------------|
| vector                       | <code>np.array([1, 2])</code>                      |
| matrix                       | <code>np.array([[1, 2], [3, 4]])</code>            |
| matrix-vector multiplication | <code>A @ x</code>                                 |
| dot product                  | <code>a @ b</code> or <code>np.dot(a, b)</code>    |
| transpose                    | <code>A.T</code>                                   |
| norm / length                | <code>np.linalg.norm(v)</code>                     |
| solve least squares          | <code>np.linalg.lstsq(X, y, rcond=None)</code>     |
| column of ones               | <code>np.ones_like(x)</code>                       |
| build design matrix          | <code>np.column_stack([x, np.ones_like(x)])</code> |

## 35. Naming conventions used so far

| Symbol                       | Meaning                                                    |
|------------------------------|------------------------------------------------------------|
| $\mathbf{x}$                 | input vector, or vector of $x$ values depending on context |
| $\mathbf{y}$                 | actual target values                                       |
| $\hat{\mathbf{y}}$           | predicted target values                                    |
| $\mathbf{r}$                 | residual vector                                            |
| $X$                          | design matrix                                              |
| $A$                          | general matrix / transformation matrix                     |
| $\boldsymbol{\beta}$         | parameter vector                                           |
| $m$                          | line slope / gradient                                      |
| $c$                          | line intercept                                             |
| $\mathbf{e}_1, \mathbf{e}_2$ | standard basis vectors                                     |

## 36. The big story so far

The notebooks have been building this chain of ideas:

1. Vectors are points, directions, and structured number collections.
2. Dot products measure alignment.
3. Projection means “the part of one vector in another direction”.
4. Matrices transform vectors.
5. Matrix columns tell us where basis vectors land.
6. Matrix-vector multiplication is a weighted combination of columns.
7. A design matrix lets a model make many predictions at once.
8. Least squares finds the prediction vector closest to the actual target vector.
9. Geometrically, least squares is projection onto the column space of the design matrix.

The key bridge into machine learning is this:

A model can be written as a matrix operation, and fitting a model can be understood as choosing parameters that make the model’s prediction vector as close as possible to the target vector.

---

## 37. Tiny worked example: predictions from a design matrix

Suppose:

$$y = mx + c$$

and:

$$m = 2, \quad c = 3$$

Collect the parameters:

$$\boldsymbol{\beta} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

For  $x = 1, 2, 3, 4$ :

$$X = \begin{bmatrix} 1 & 1 & 2 & 1 & 3 & 1 & 4 & 1 \\ \end{bmatrix}$$

Predictions:

$$\hat{\mathbf{y}} = X\boldsymbol{\beta}$$

$$\hat{\mathbf{y}} = \begin{bmatrix} 1 & 1 & 2 & 1 & 3 & 1 & 4 & 1 \\ \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 & 7 & 9 & 11 \\ \end{bmatrix}$$

The matrix has evaluated the line at all four  $x$  values in one operation.

---

### 38. Tiny worked example: residuals

Suppose the actual values are:

$$\mathbf{y} = \begin{bmatrix} 6 \\ 6 \\ 10 \\ 12 \end{bmatrix}$$

and the predictions are:

$$\hat{\mathbf{y}} = \begin{bmatrix} 5 \\ 7 \\ 9 \\ 11 \end{bmatrix}$$

Then residuals are:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$$

$$\mathbf{r} = \begin{bmatrix} 6 \\ 6 \\ 10 \\ 12 \end{bmatrix}$$

$$- \begin{bmatrix} 5 \\ 7 \\ 9 \\ 11 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix}$$

Sum of squared residuals:

$$1^2 + (-1)^2 + 1^2 + 1^2 = 4$$

---

### 39. Useful mental models

#### Dot product

How much do these point the same way?

#### Projection

What part of this vector lies along that direction?

#### Matrix

Where do the basis vectors go?

#### Matrix-vector multiplication

Build the transformed vector by mixing the transformed basis vectors.

## Design matrix

Arrange the cases where the model should be evaluated.

## Least squares

Find the reachable prediction vector closest to the target vector.

## Column space

The set of all outputs the matrix can produce.

---

## 40. Things to be careful about

### 1. Dot product is not vector multiplication

The dot product produces a scalar, not another vector.

### 2. Order matters in matrix multiplication

Usually:

$$AB \neq BA$$

### 3. A design matrix is context-dependent

In model evaluation, it represents the cases where we want predictions.

In training/fitting, those cases may come from training data.

The matrix itself is not magically “training”; it depends what we are doing with it.

### 4. Residual sign convention matters

We are using:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$$

Some resources use the opposite convention. The squared residuals are the same either way, but the signs differ.

### 5. Avoid manually inverting matrices unless there is a reason

For least squares, prefer:

```
np.linalg.lstsq(X, y, rcond=None)
```

rather than explicitly calculating:

```
np.linalg.inv(X.T @ X) @ X.T @ y
```

The latter is useful for understanding the formula, but not usually the best computational method.